Juniorprogrammierer.de

Go 4: Error handling

2024/25/26 – Sascha Stojanovic

Errors = Objects satisfying error-interface

```
// type error interface {
//          Error() string
// }

type AgeError struct {
          Age int
}

func (e AgeError) Error() string {
          return fmt.Sprintf("Alter %d ist zu klein - muss
          mindestens 18 sein", e.Age)
}

func checkAge(age int) error {
          if age < 18 {
                    Return AgeError{Age: age}
          }
          return nil
}
```

```
func main() {
        err := checkAge(15)
        if err != nil {
                fmt.Println("Fehler:", err)} // err.Error()
        err = checkAge(20)
        if err != nil {
                fmt.Println("Fehler:", err)
        } else {
                fmt.Println("Alter ist OK!")
        }
}
```

Errors in go -- Details

```
// type error interface {
//         Error() string
// }

type AgeError struct {
        Age int
}

func (e AgeError) Error() string {
        return fmt.Sprintf("Alter %d ist zu klein - muss
        mindestens 18 sein", e.Age)
}

func checkAge(age int) error {
        if age < 18 {
                Return AgeError{Age: age}
        }
        return nil
}
```

Comment or not doesn't matter

If no Error always return nil

err == err.Error()

```
func main() {
        err := checkAge(15)
        if err != nil {
                fmt.Println("Fehler:", err)}
        err = checkAge(20)
        if err != nil {
                fmt.Println("Fehler:", err)
        } else {
                fmt.Println("Alter ist OK!")
        }
}
```

Inline-error creation

- Go offers also a fast creation of an Error object with custom text
- What is the advantage of a custom error creation compared to the inline variant?

```
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("Division durch 0!")
    }
    return a / b, nil
}
```

Exercize

1. Create a custom error type called WithdrawalError with these fields:
   - Balance (current balance)
   - Amount (amount trying to withdraw)
   - Reason (string: why it failed)
2. Implement the Error() method that returns a message like:

- Cannot withdraw $150. Balance is $100. Reason: insufficient funds

3. Create a withdraw(balance, amount int) function that returns error:
   - If amount <= 0 → return WithdrawalError with reason "amount must be positive"
   - If amount > balance → return WithdrawalError with reason "insufficient funds"
   - If amount > 1000 → return WithdrawalError with reason "daily limit exceeded"
   - Otherwise → return nil (success)
4. In main(), test these cases:
   - Withdraw $50 from balance $100 (should succeed)
   - Withdraw $-20 from balance $100 (should fail)
   - Withdraw $150 from balance $100 (should fail)
   - Withdraw $1500 from balance $2000 (should fail)

Expected Output:

Success! Withdrew $50

Error: Cannot withdraw $-20. Balance is $100. Reason: amount must be positive

Error: Cannot withdraw $150. Balance is $100. Reason: insufficient funds

Error: Cannot withdraw $1500. Balance is $2000. Reason: daily limit exceeded